Predicting At-Risk Students in Online Learning: A Comparison of Traditional Machine Learning and Deep Learning on the Open University Learning Analytics Dataset

Aderline Gashugi, African Leadership University, Machine Learning Summative Project

GitHub repository: github.com/Aderline490/oulad-at-risk-students
Google Drive folder (raw dataset and saved models): Drive folder
Video presentation:https://youtu.be/4xlPRikRx8Q

Abstract: Many students who start online courses do not finish them, and tutors can only help if they know early who is struggling. In this project I used the Open University Learning Analytics Dataset (32,593 student-course records and 10.6 million click records) to predict which students will fail or withdraw, using only data from the first 28 days of a course. After removing students who had already left by day 28, 27,535 records were left, and 44% of them were at risk. I ran 19 experiments comparing scikit-learn models with Keras neural networks trained with tf.data. My engineered features raised the ROC-AUC of logistic regression from 0.660 to 0.773. The best traditional model (gradient boosting, test AUC 0.792) and the best deep model (an MLP with a GRU branch, 0.790) were statistically tied. With a lower threshold, both models found about 80% of at-risk students.

Index Terms: learning analytics, dropout prediction, early warning, gradient boosting, deep learning, OULAD.

I. Introduction

Online learning has made higher education possible for many people who cannot attend a campus, including many students in Africa. The problem is that a lot of online students never finish. Simpson [1] calls this the "distance education deficit", and Lee and Choi [2] reviewed many studies on why online learners drop out. Both agree that contacting struggling students early can help, but only if the institution knows who they are.

This topic connects to my personal mission. I want to empower and uplift children in orphanages across Africa by creating opportunities in education, emotional support and long-term self-reliance, and by connecting them with reliable families, so that every child can grow up with dignity, hope and a sense of belonging. Many of these children have nobody at home who notices when they start falling behind in school, so they can drop out quietly. A tool that flags struggling learners early could make sure that someone reaches out to them in time. I cannot get data on children in orphanages, so in this project I use a large public dataset from online learning to understand how early-warning models work and where they fail, especially for students from poorer backgrounds.

In this project I predict whether a student is **at risk** (final result Fail or Withdrawn) or **not at risk** (Pass or Distinction). The models can only use information from **before day 28** of the course: registration details, clicks in the virtual learning environment (VLE) during the first four weeks, and any coursework due in that time. A prediction made at the end of the course would be more accurate, but by then it is too late to help.

I used the Open University Learning Analytics Dataset (OULAD) [3], which is public under a CC BY 4.0 licence and available on the UCI Machine Learning Repository. It has 32,593 student-course records from 7 modules and 22 course presentations, 10,655,280 rows of daily click data and 173,912 assessment submissions. I chose it because it is real data from a large distance-learning university, it is detailed enough for both engineered features and sequence models, and many researchers have used it, so I can compare my work with theirs. Its main weakness is that it comes from the UK and not from Africa.

My objectives were to build a pipeline that does not leak future information, to compare traditional machine learning and deep learning on exactly the same data, and to look past accuracy by studying learning curves, ROC curves, confusion matrices and the errors the model makes for different groups of students.

II. Literature Review

A. Why students drop out

Lee and Choi [2] grouped the reasons for online dropout into student factors (such as prior education), course factors (such as course design) and environmental factors (such as work and family). Simpson [1] argued that universities spend a lot on teaching materials but not enough on motivating students, and that contacting students early is what actually reduces dropout. Kizilcec, Piech and Schneider [4] found that MOOC learners follow different engagement patterns, for example completing, auditing or slowly disengaging. This suggested to me that the way a student's activity changes over time could be useful, and not only how much they click in total. From these studies I understood that an early-warning model is only useful if it flags students early enough for someone to act.

B. Predictive models on OULAD

Kuzilek, Hlosta and Zdrahal [3] released OULAD as a benchmark for this kind of research. The same group built Ouroboros [5], which trains on the current course instead of older runs, because a model trained on last year's course may not work if the course changes. Adnan et al. [6] compared several traditional algorithms at different points in the course and found that Random Forest worked best, with accuracy improving as more weeks were observed. This showed me that the prediction point has to be fixed when comparing models, otherwise the model that sees more weeks will always look better.

Several studies used deep learning on OULAD. Waheed et al. [7] trained a deep neural network on clickstream features and beat logistic regression and SVM baselines. Hassan et al. [8] and Aljohani, Fayoumi and Hassan [9] used LSTMs on weekly activity, and [9] reports around 90% accuracy using the first ten weeks. These results look strong, but I noticed two problems when comparing them. The windows are long, since ten weeks is a big part of a course. Also, the baselines are mostly simple linear models and not tuned tree models, so it is not clear if deep learning is really better for this data.

C. Deep learning versus trees on tabular data

Outside education, Grinsztajn, Oyallon and Varoquaux [10] tested models on 45 tabular datasets and found that tree-based models still beat deep learning on medium-sized tabular data. Shwartz-Ziv and Armon [11] found that XGBoost usually matched or beat deep tabular models and needed less tuning. Once OULAD is summarised per student, it becomes this kind of tabular data. The only part that is not

tabular is the daily click sequence, which a convolutional or recurrent network could read directly. I did not find many OULAD studies that compare a tuned tree model and a sequence network on the same features and the same cutoff.

D. Evaluation and fairness

Gardner and Brooks [12] warned that many dropout studies are hard to reproduce and that some use features which would not exist yet at prediction time. Saito and Rehmsmeier [13] showed that precision-recall curves are more informative than ROC curves when the positive class is the smaller one, which is relevant here since 44% of my cohort is at risk. Gardner, Brooks and Baker [14] proposed checking model performance separately for each group of students, and Baker and Hawn [15] reviewed evidence that educational algorithms can be biased by gender, ethnicity and income.

E. The gap this project addresses

Based on these studies, my project tries to fill four gaps for this dataset. I fix the prediction point at day 28 and remove students whose outcome was already known. I split the data by student so the same person is never in both training and test. I compare tuned tree models, an SVM and logistic regression with several neural networks, including ones that read the daily clicks, all on the same data. Finally, I check how the errors are spread across courses and student groups, following [14].

III. Methodology

A. Dataset and cohort

OULAD has seven linked tables, including demographics and final results, registration dates, assessments and the daily click log. One student taking one course presentation is one row in my data. I checked the raw tables and found no duplicates, no negative click counts and no scores outside 0 to 100.

I removed two groups of students. The first is the 5,040 students who had already withdrawn before day 28. Their result is already known when the prediction is made, so keeping them would make the models look better than they really are. The second is 18 students who registered on or after day 28. This left **27,535** records, and 44.1% of them are at risk (7,037 Fail and 5,119 Withdrawn, against 12,356 Pass and 3,023 Distinction). Since the imbalance is small, I did not resample. Instead I reported recall, F1 and precision-recall curves, and tested class weights in one experiment.

B. Feature engineering

The click log has more than ten million rows, so I wrote functions that turn it into one row per student with 43 features in three groups (Table I). Every feature only uses records from before day 28.

Table I. Engineered features. All use only records dated before day 28.

Group	Columns	Examples
Registration	12	course module, start month, gender, region, highest education, deprivation (IMD) band, age band, disability, previous attempts, credits, registration day, course length

Group	Columns	Examples
VLE activity	25	total clicks, active days, distinct resources, clicks before the course starts, clicks in each of weeks 1 to 4, clicks on 10 activity types, first and last active day, active weeks, click trend (log of week 4 minus log of week 1)
Assessment	6	assessments due, number submitted, share submitted, mean score, late submissions, mean days submitted early

I also kept the daily click counts from day -14 to day 27 (42 values per student) for the networks that can read sequences. I wanted to see if the day-by-day pattern adds anything to my weekly totals.

C. Preprocessing

All preprocessing is inside a scikit-learn [16] pipeline that is fitted on the training set only, so nothing is learned from the validation or test data. Categorical columns are one-hot encoded, and missing `imd_band` values (3.7%) get their own "Unknown" category instead of a guessed value. Count features are imputed with the median, log-transformed and standardised. I needed the log transform because the counts are very skewed: total clicks has a median of 221 and a maximum of 7,570, and the skew drops from 3.48 to -1.37 after the transform. I did not delete the extreme values because they are real students. Other numeric columns get median imputation with a missing-value indicator. The indicator is useful for `mean_score`, which is missing for 27% of students, and more often for at-risk students (33.7% against 22.3%). After preprocessing each student has 85 inputs.

D. Data split

I split the data 70/15/15 into training (19,293 rows), validation (4,117) and test (4,125). The split is grouped by student ID, because 3,538 students appear in more than one course and I did not want the same person in training and test. The at-risk share is almost the same in all three sets (44.2%, 43.9% and 44.3%). Hyperparameter searches used 3-fold grouped cross-validation on the training set. I used the validation set to compare experiments and choose thresholds, and the test set only for the final numbers.

E. Traditional machine learning models

I trained Logistic Regression as a simple baseline, Random Forest [17], gradient boosting with `HistGradientBoostingClassifier` (which follows the LightGBM approach [18]) and an SVM with an RBF kernel. I tuned them with grid search or random search, scored by ROC-AUC.

F. Deep learning models

The networks were built in TensorFlow/Keras [19] and trained with `tf.data` pipelines that cache, shuffle, batch (256 rows) and prefetch the data. The **Sequential** models are dense networks (64 and 32 units by default) with ReLU and optional batch normalisation [20], dropout [21] and L2 regularisation. The **Functional** models have two inputs: the same dense branch for the 85 features, and a second branch for the 42-day sequence that uses either 1D convolutions or a 32-unit GRU [22]. The two branches are joined before the output layer. All models use binary cross-entropy and, unless stated otherwise, Adam [23] with a learning rate of 0.001 and early stopping on validation AUC with a patience of 8 epochs.

G. Experiment design and evaluation

I ran 19 experiments, 10 traditional (ML-01 to ML-10) and 9 deep (DL-01 to DL-09), where each one tests a question raised by the one before. Every run is logged in TensorBoard under its ID, so each number in Table II can be traced back to a run. I report accuracy, precision, recall, F1, ROC-AUC and PR-AUC for the at-risk class. I set the random seed to 42, and running the notebook twice gave identical results. For the final models I also made learning curves, ran a paired bootstrap test (1,000 resamples), chose a threshold on validation that reaches 80% recall, and analysed the errors by final result, course and student group [14].

IV. Results

A. What the data shows before modelling

Fig. 1 shows that early behaviour already separates the two classes. At-risk students click less on all 42 days, even before the course starts, and their median total is 150 clicks against 293. The at-risk rate drops from 82% for the 922 students with no activity to 27% for students active on 21 days or more. It also changes a lot by course (25.9% in module AAA against 52.3% in CCC) and by deprivation band (56.1% in the most deprived areas against 34.9% in the least deprived).

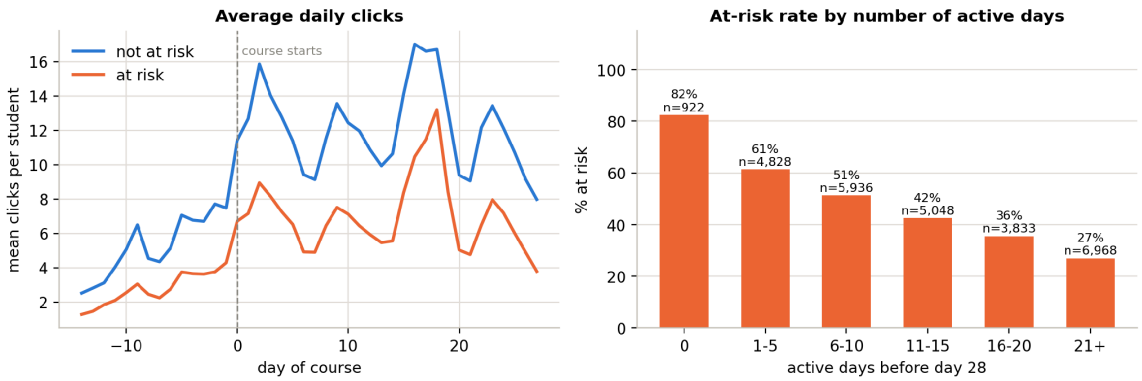


Fig. 1. Average daily clicks by class (left) and at-risk rate by number of active days before day 28 (right).

B. Experiment table

Table II shows all 19 experiments. They all use the same split and the same metrics, and each ID matches a TensorBoard log folder. Recall and F1 are for the at-risk class at the default threshold. The TensorBoard logs behind these runs are shown in Appendix A.

Table II. All experiments. AUC is ROC-AUC. Split for every row: 70/15/15 grouped by student.

ID	Model	Key settings	Train AUC	Val AUC	Val F1	Val recall	Test AUC	Observation
ML-01	Logistic Regression	registration features only (12), C=1	0.665	0.660	0.508	0.445	0.654	Weak baseline: who the student is says little

ID	Model	Key settings	Train AUC	Val AUC	Val F1	Val recall	Test AUC	Observation
ML-02	Logistic Regression	+ VLE features (37), C=1	0.758	0.748	0.618	0.572	0.756	Biggest single gain (+0.088)
ML-03	Logistic Regression	all 43 features, C=1	0.784	0.773	0.655	0.605	0.780	Assessment adds +0.025; train and val close, so underfitting
ML-04	Logistic Regression (tuned)	grid of 14: best C=10, L2	0.784	0.773	0.654	0.603	0.781	No gain: regularisation is not the bottleneck
ML-05	Random Forest	200 trees, unlimited depth	1.000	0.773	0.644	0.583	0.777	Memorises training set (high variance)
ML-06	Random Forest (tuned)	random search of 12: depth 12, leaf 5, max_features 0.5, 400 trees	0.949	0.777	0.643	0.578	0.781	Depth limit helps, gap still 0.17
ML-07	Hist Gradient Boosting	lr 0.1, 100 trees, 31 leaves, early stopping	0.861	0.783	0.659	0.611	0.791	Best family so far, trains in about 1 s
ML-08	Hist Gradient Boosting (tuned)	random search of 15: lr 0.05, 31 leaves, min leaf 100, L2 1.0, up to 800 trees	0.842	0.783	0.662	0.616	**0.792**	Same val score with a smaller gap; best traditional
ML-09	SVM, RBF (tuned)	grid of 6: C=3, gamma 0.003	0.803	0.782	0.639	0.557	0.788	Competitive but about 30 s and lowest recall

ID	Model	Key settings	Train AUC	Val AUC	Val F1	Val recall	Test AUC	Observation
ML-10	HGB + feature selection	ML-08 settings, top 20 by mutual information	0.829	0.749	0.628	0.579	0.759	Kept 18 correlated click counts and dropped module and education: -0.034
DL-01	MLP 64-32	no regularisation , 40 epochs, no early stopping	0.881	0.758	0.640	0.610	0.754	Clear overfitting after epoch 9
DL-02	MLP 64-32	+ batch norm, dropout 0.3, early stopping	0.823	0.784	0.657	0.608	0.788	Regularisation adds +0.026 and beats every ML model on val
DL-03	MLP 128-64-32	+ L2 1e-4	0.818	0.782	0.659	0.609	0.792	More capacity does not help
DL-04	MLP 64-32	Adam lr 0.01	0.818	0.784	0.652	0.596	0.787	Stops at epoch 19, noisier validation loss
DL-05	MLP 64-32	Adam lr 0.0001	0.799	0.776	0.656	0.616	0.784	Still improving at epoch 60, so it underfits
DL-06	MLP 64-32	SGD + momentum 0.9, lr 0.01	0.811	0.782	0.660	0.610	0.789	Also needs all 60 epochs
DL-07	Functional: MLP + Conv1D	daily click sequence branch	0.819	0.785	0.665	0.621	0.790	Sequence adds about +0.001
DL-08	Functional: MLP + GRU	daily click sequence branch	0.817	0.786	0.655	0.590	0.790	Best single architecture, 3 times slower than Conv1D

ID	Model	Key settings	Train AUC	Val AUC	Val F1	Val recall	Test AUC	Observation
DL-09	Functional: MLP + GRU	+ class weights, ReduceLROnPlateau	0.816	**0.786**	**0.672**	**0.657**	0.790	Same ranking, better recall and F1 at 0.5; best deep

Fig. 2 shows the validation scores side by side. Most of the good models are very close: 15 of the 19 runs are between 0.772 and 0.787.

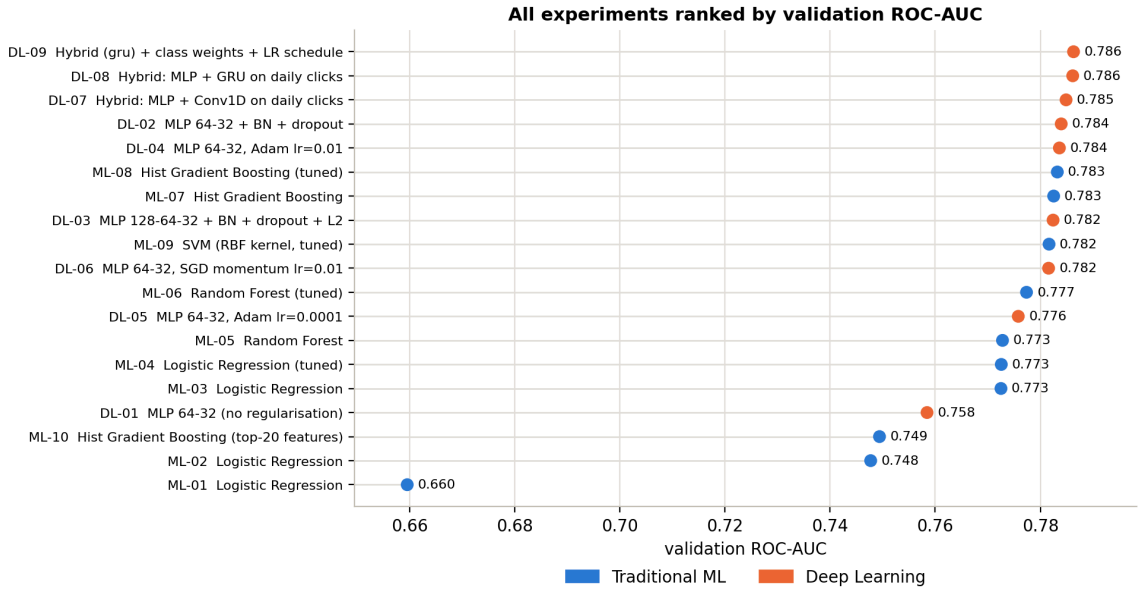


Fig. 2. All 19 experiments ranked by validation ROC-AUC.

C. Learning curves

Fig. 3 shows the loss curves of the nine networks. DL-01 overfits: its validation loss is lowest at epoch 9 (0.549) and then rises to 0.606, while training loss keeps dropping to 0.437. The regularised networks keep the two curves close, with validation loss levelling off around 0.55 to 0.56. DL-05 and DL-06 are still going down at epoch 60, and DL-04 has the noisiest validation curve.

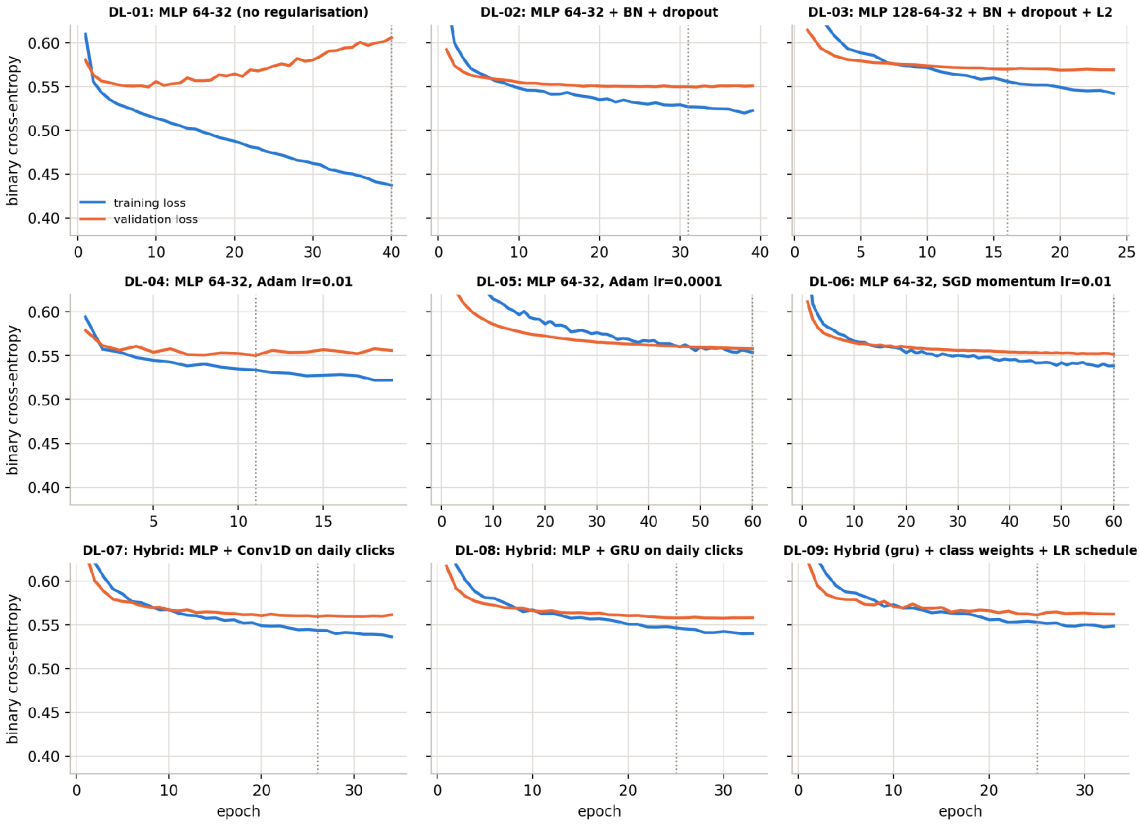


Fig. 3. Training and validation loss per epoch for DL-01 to DL-09. The dotted line marks the epoch whose weights were kept.

Fig. 4 shows the learning curve of ML-08. The cross-validated AUC is 0.757 with 1,286 rows and stops improving at about 0.788 after around 10,500 rows, while the gap to the training score stays near 0.08.

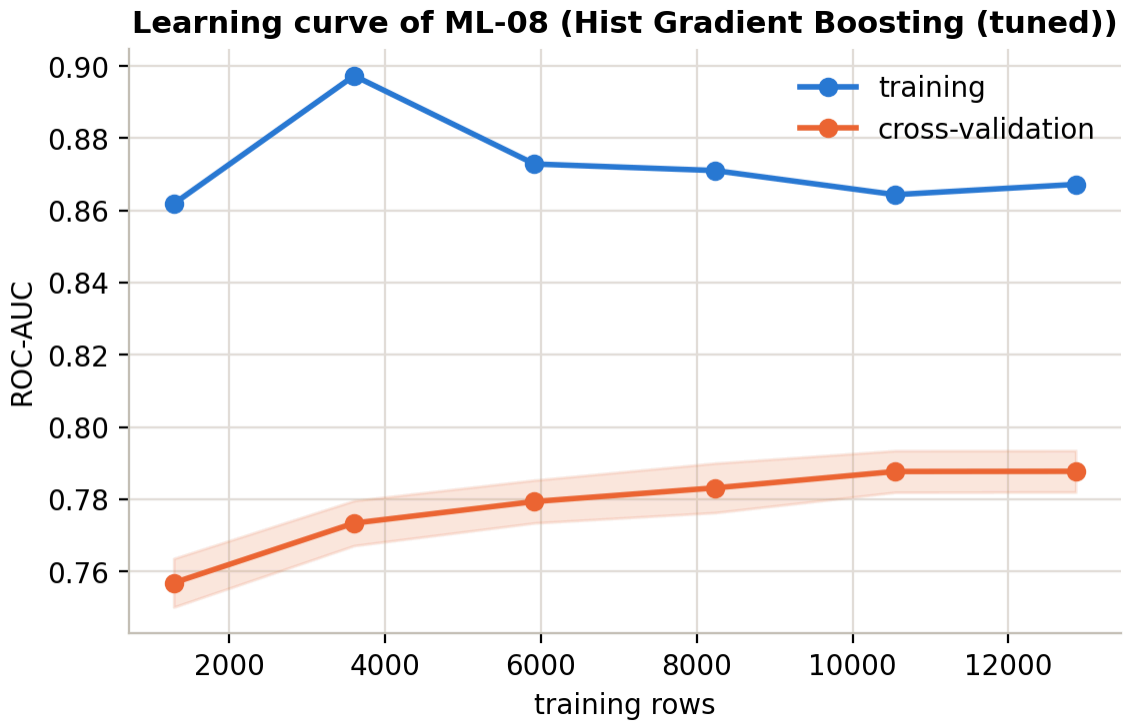


Fig. 4. Learning curve of ML-08 with 3-fold grouped cross-validation.

D. Final models on the test set

In Fig. 5 the best models from each family have almost the same curves. On the test set ML-08 has an AUC of 0.792 and DL-09 has 0.790, even though DL-09 was ahead on validation. A paired bootstrap with 1,000 resamples gave a difference of 0.002, with a 95% interval from -0.003 to +0.007. Since the interval includes zero, the two models are tied.

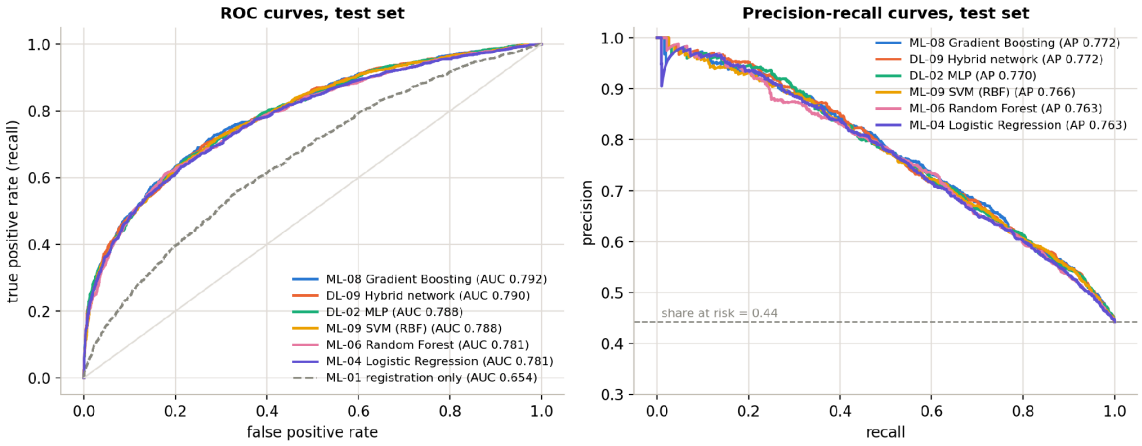


Fig. 5. Test-set ROC curves (left) and precision-recall curves (right) for the best model of each family.

At the default threshold of 0.5, ML-08 misses 688 of the 1,826 at-risk students in the test set (Fig. 6). So I chose the highest threshold that still reaches 80% recall on validation (0.331 for ML-08 and 0.378 for DL-09) and used it on the test set (Table III).

Table III. Test-set results at the default and the recall-oriented threshold.

Model	Threshold	Accuracy	Precision	Recall	F1
ML-08	0.500	0.724	0.717	0.623	0.667
ML-08	0.331	0.679	0.603	0.807	0.690
DL-09	0.500	0.721	0.695	0.659	0.677
DL-09	0.378	0.684	0.610	0.795	0.690

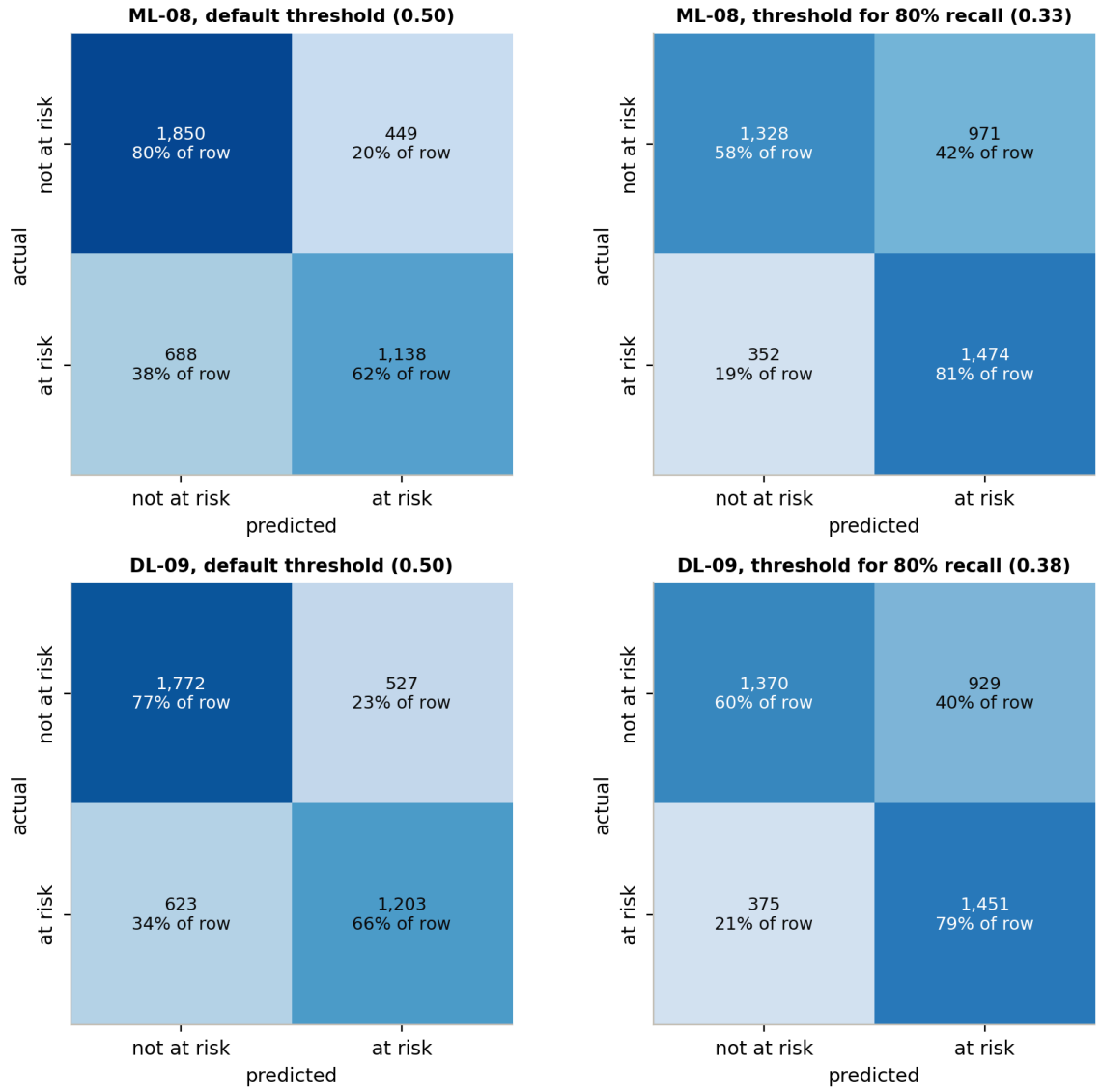


Fig. 6. Test-set confusion matrices of ML-08 and DL-09 at the default threshold and at the threshold chosen for 80% recall.

Permutation importance (Fig. 7) ranks the early assessment score first (a 0.068 drop in AUC when shuffled), then course module (0.034) and previous education (0.015). By group, the assessment features add up to 0.103, registration to 0.064 and VLE activity to only 0.016.

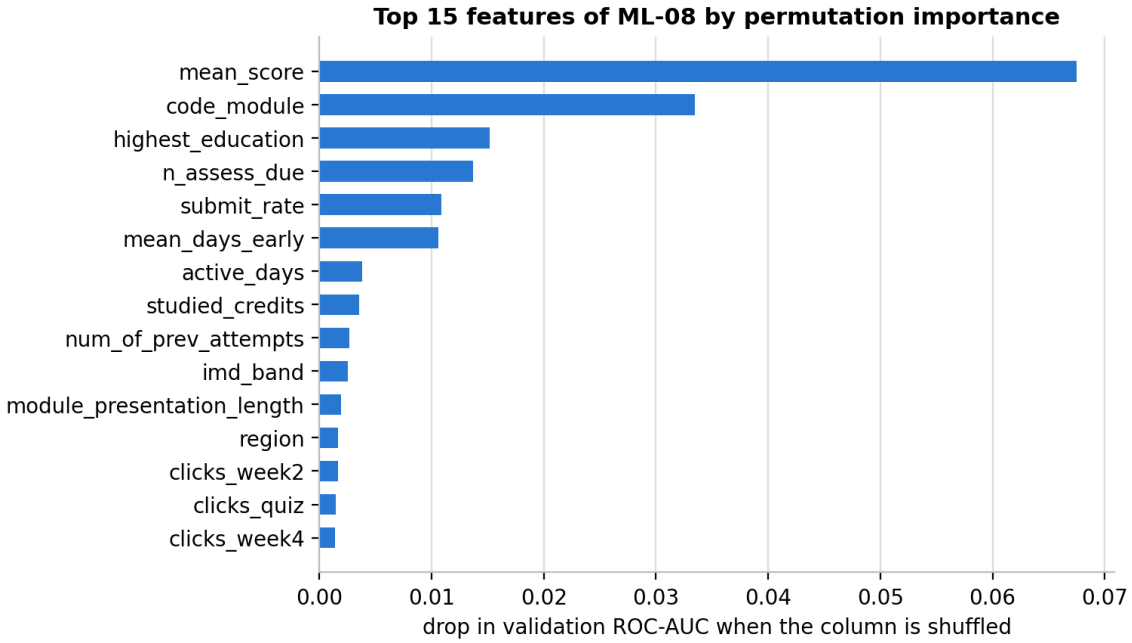


Fig. 7. Top 15 features of ML-08 by permutation importance on the validation set.

E. Errors by group

I looked at the errors of DL-09 at its 0.378 threshold. It finds 80.8% of the students who fail and 77.6% of those who withdraw, but it also flags 44.5% of Pass students and 22.8% of Distinction students. Table IV shows that performance also changes between groups.

Table IV. Test-set performance of DL-09 at threshold 0.378 for selected groups.

Group	Students	% at risk	Recall	False positive rate	AUC
Module DDD	772	52.1	0.876	0.422	0.833
Module GGG	381	39.9	0.704	0.511	0.656
Most deprived (IMD 0-30%)	1,318	52.8	0.874	0.529	0.799
Least deprived (IMD 70-100%)	1,066	38.4	0.709	0.326	0.767
Female	1,893	43.5	0.765	0.411	0.766
Male	2,232	44.9	0.819	0.398	0.810

Group	Students	% at risk	Recall	False positive rate	AUC
Declared disability	386	54.7	0.929	0.629	0.777
No declared disability	3,739	43.2	0.777	0.386	0.790

V. Discussion

A. Features mattered more than the model

The biggest lesson for me is that the inputs mattered more than the model. Adding my engineered features raised the AUC of the same logistic regression by 0.113, while going from the best logistic regression to the best deep model only added about 0.014. This agrees with [10] and [11]. After I summarised the click data into one row per student, it became a normal tabular problem, and tree models are hard to beat on that kind of data.

The sequence models point the same way. Reading the 42 daily values added only 0.002 AUC with the GRU and 0.001 with Conv1D, probably because my weekly features already contain most of that information. The GRU also took three times longer to train. I think this is why deep learning looks stronger in [7]-[9]: those studies used longer windows and simpler baselines. With four weeks of data and a tuned gradient boosting model, I did not find a real advantage. If I had only looked at validation AUC I would have said the deep model won, but the bootstrap shows the difference is not meaningful. For real use I would choose gradient boosting, because it trains in about 8 seconds and is easier to explain to tutors.

B. Hyperparameters and bias-variance

The experiments show both sides of the bias-variance trade-off. Logistic regression has high bias: its training and validation AUC were within 0.011, and changing C from 0.01 to 10 made no difference. The default Random Forest has high variance, with a training AUC of 1.000 against 0.773 on validation. Limiting the depth and leaf size in ML-06 reduced that gap from 0.227 to 0.171. Gradient boosting was the most balanced, and the tuned version (at least 100 students per leaf, plus L2) lowered training AUC from 0.861 to 0.842 without losing any validation AUC.

For the networks, DL-01 overfit after epoch 9 because nothing stopped it. Adding batch normalisation, dropout and early stopping in DL-02 improved validation AUC by 0.026. A learning rate of 0.01 made the validation loss jump around because the steps were too big, and 0.0001 was too slow to finish learning in 60 epochs. SGD with momentum behaved like a slower version of Adam [23].

The learning curve in Fig. 4 shows that more data of the same kind would not help much, because the curve has flattened and the gap stays around 0.08. Most of the remaining error comes from things that four weeks of data cannot show. My first version of this plot looked very different. The data was sorted by course, so the small training subsets only had one or two modules in them. After I shuffled the data the conclusion changed, and I learned to check how a plot samples the data before trusting it.

C. Error analysis

Most false positives are students who end with a Pass (44.5% of them were flagged). After only four weeks, a student who passes narrowly can look the same as one who fails. The false negatives are different. Their median activity (350 clicks, 16 active days, early score of 82) is almost the same as that of students who were fine (407 clicks, 19 days, score of 85). The withdrawn students the model missed left on day 125 (median), while the ones it caught left on day 101. These students were doing fine in the first month and something changed later, which no day-28 model can see.

Lowering the threshold from 0.5 to 0.33 raised ML-08's recall from 62% to 81%, but it more than doubled the false alarms, from 449 to 971. The precision-recall curve [13] shows this trade-off better than accuracy does. At 80% recall, about 4 out of 10 flagged students were actually fine. That is acceptable for a friendly email, but maybe not for a phone call from a busy tutor.

Permutation importance (Fig. 7) makes the VLE features look unimportant, even though they gave the biggest gain in my ablation (+0.088). This is because the 25 VLE columns are strongly correlated, so when one is shuffled the model just uses the others. The same reason explains why feature selection failed in ML-10: it kept 18 similar click columns and dropped course and education.

D. Fairness and limitations

Table IV shows problems that the overall AUC hides [14], [15]. With one threshold, 52.9% of the students from the most deprived areas who were actually fine got flagged, compared with 32.6% in the least deprived areas. Part of this comes from their higher at-risk rate, but it still means poorer students would be contacted more often. The gender gap worries me more. The model ranks men better than women (AUC 0.810 against 0.766) even though their at-risk rates are almost the same. I kept these features, because removing them would not remove the bias when region, course and clicks carry similar information. Checking these numbers regularly is a better solution. For my mission this part matters a lot, because the students I care about most are the ones a biased model could serve worst.

The dataset also has limits. It comes from one UK university in 2013 and 2014, and the deprivation bands are UK measures. Students in African online programmes may have different internet access, devices and family responsibilities, so I would retrain the model before using it at a school like ALU. Clicks also do not show learning, since a student who downloads everything and studies offline looks inactive. Merging Fail and Withdrawn hides the fact that these students may need different help, and the model works much worse in module GGG (AUC 0.656) than in DDD (0.833).

E. Future work

Since many missed students drop out late, I would re-score students every two to four weeks as new data comes in. I would also test the model on a later year (train on 2013 and test on 2014), try separate thresholds for weaker courses like GGG, and calibrate the probabilities so that tutors can trust the scores.

VI. Conclusion

In this project I built a model that predicts, from the first 28 days of an online course, which students will fail or withdraw. I removed students whose outcome was already known, split the data by student, and

fitted all preprocessing on the training data only. Across 19 experiments, the best traditional model (gradient boosting) and the best deep model (MLP with a GRU) both reached a test AUC of about 0.79, and the difference between them was not significant. With a lower threshold, both find about 4 in 5 at-risk students, but about 4 in 10 flagged students are false alarms.

The main thing I learned is that good features and honest evaluation mattered more than the choice of model. The neural networks needed regularisation just to match a model that trains in seconds, and the daily click sequence added almost nothing. Most of the remaining errors are students who drop out months later, which a day-28 model cannot see. If I used this in practice, I would choose gradient boosting, re-score students every few weeks and check the results for different groups. Before using anything like this for the children my mission is about, I would need local data and a lot of care, because a wrong flag affects a real child. Still, this project showed me that early warning is possible with simple data, and that checking who a model fails is as important as its accuracy.

Appendix A. TensorBoard Logs

Fig. A1 shows the HParams table in TensorBoard with the settings and final metrics of the experiments, and Fig. A2 shows the per-epoch accuracy and AUC curves of the Keras runs. TensorBoard gives each run its own trial ID, but the run folders in the `logs/` folder of the GitHub repository are named by experiment ID (ML-01 to DL-09), so every number in Table II can be checked against these logs.

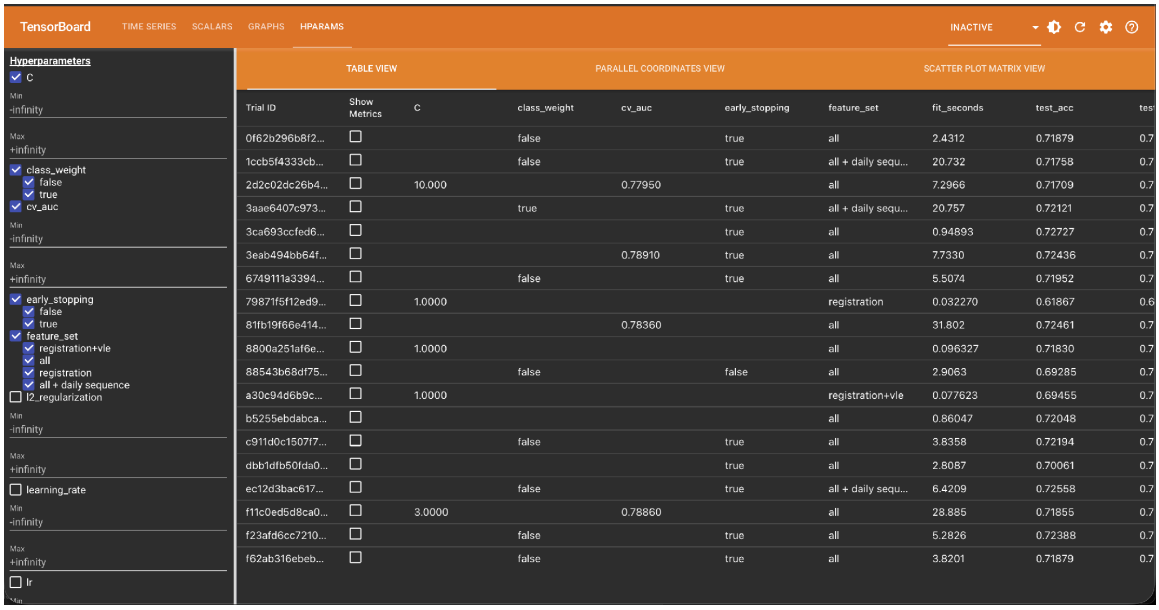


Fig. A1. TensorBoard HParams table view with the settings and final metrics of the logged experiments.



Fig. A2. TensorBoard per-epoch accuracy and AUC curves for the Keras runs DL-01 to DL-09.

References

[1] O. Simpson, "Student retention in distance education: are we failing our students?," *Open Learning: The Journal of Open, Distance and e-Learning*, vol. 28, no. 2, pp. 105–119, 2013, doi: 10.1080/02680513.2013.847363.

[2] Y. Lee and J. Choi, "A review of online course dropout research: implications for practice and future research," *Educational Technology Research and Development*, vol. 59, no. 5, pp. 593–618, 2011.

[3] J. Kuzilek, M. Hlosta, and Z. Zdrahal, "Open University Learning Analytics dataset," *Scientific Data*, vol. 4, Art. no. 170171, 2017, doi: 10.1038/sdata.2017.171.

[4] R. F. Kizilcec, C. Piech, and E. Schneider, "Deconstructing disengagement: analyzing learner subpopulations in massive open online courses," in *Proc. 3rd Int. Conf. Learning Analytics and Knowledge (LAK '13)*, Leuven, Belgium, 2013, pp. 170–179.

[5] M. Hlosta, Z. Zdrahal, and J. Zendulka, "Ouroboros: early identification of at-risk students without models based on legacy data," in *Proc. 7th Int. Learning Analytics and Knowledge Conf. (LAK '17)*, Vancouver, Canada, 2017, pp. 6–15, doi: 10.1145/3027385.3027449.

[6] M. Adnan *et al.*, "Predicting at-risk students at different percentages of course length for early intervention using machine learning models," *IEEE Access*, vol. 9, pp. 7519–7539, 2021, doi: 10.1109/ACCESS.2021.3049446.

[7] H. Waheed, S.-U. Hassan, N. R. Aljohani, J. Hardman, and R. Nawaz, "Predicting academic performance of students from VLE big data using deep learning models," *Computers in Human Behavior*, vol. 104, Art. no. 106189, 2020, doi: 10.1016/j.chb.2019.106189.

[8] S.-U. Hassan, H. Waheed, N. R. Aljohani, M. Ali, S. Ventura, and F. Herrera, "Virtual learning environment to predict withdrawal by leveraging deep learning," *International Journal of Intelligent Systems*, vol. 34, no. 8, pp. 1935–1952, 2019, doi: 10.1002/int.22129.

[9] N. R. Aljohani, A. Fayoumi, and S.-U. Hassan, "Predicting at-risk students using clickstream data in the virtual learning environment," *Sustainability*, vol. 11, no. 24, Art. no. 7238, 2019.

[10] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022.

[11] R. Shwartz-Ziv and A. Armon, "Tabular data: deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[12] J. Gardner and C. Brooks, "Student success prediction in MOOCs," *User Modeling and User-Adapted Interaction*, vol. 28, no. 2, pp. 127–203, 2018.

[13] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015.

[14] J. Gardner, C. Brooks, and R. Baker, "Evaluating the fairness of predictive student models through slicing analysis," in *Proc. 9th Int. Conf. Learning Analytics and Knowledge (LAK '19)*, Tempe, AZ, USA, 2019, pp. 225–234, doi: 10.1145/3303772.3303791.

[15] R. S. Baker and A. Hawn, "Algorithmic bias in education," *International Journal of Artificial Intelligence in Education*, vol. 32, no. 4, pp. 1052–1092, 2022.

[16] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[18] G. Ke *et al.*, "LightGBM: a highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017, pp. 3146–3154.

[19] M. Abadi *et al.*, "TensorFlow: a system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI '16)*, Savannah, GA, USA, 2016, pp. 265–283.

[20] S. Ioffe and C. Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, Lille, France, 2015, pp. 448–456.

[21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[22] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. 2014 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1724–1734.

[23] D. P. Kingma and J. Ba, "Adam: a method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.